

Задание

Разработать программу для системы мониторинга автоматизированной теплицы

В теплице установлены датчики температуры, влажности и освещенности. Каждый датчик хранит историю последних измерений и умеет добавлять новое измерение в историю

В программе должны быть созданы следующие датчики:

Идентификатор	Тип измерения	Модель	Размер истории	Файл измерений
TEMP_AIR_01	температура воздуха	AirTempT100	8	temp_air_01.txt
TEMP_AIR_02	температура воздуха	AirTempT100	8	temp_air_02.txt
HUM_AIR_01	влажность воздуха	HumidityH200	10	humidity_air_01.txt
LIGHT_EAST_01	освещенность восточной грядки	LightLX300	12	light_east_01.txt
LIGHT_WEST_01	освещенность западной грядки	LightLX300	12	light_west_01.txt

Для каждого датчика необходимо хранить:

- идентификатор
- тип измеряемой величины
- модель датчика
- историю последних измерений
- профиль калибровки

Профиль калибровки содержит название модели датчика, смещение измерения и коэффициент масштаба. Несколько датчиков одной модели должны использовать один общий профиль калибровки

Использовать следующие профили калибровки:

Модель	Смещение	Масштаб
AirTempT100	-0.4	1.02
HumidityH200	1.5	0.98
LightLX300	0.0	1.10

Добавление измерения должно применять калибровку по правилу:

```
calibrated = raw * scale + offset
```

История хранит только последние измерения в количестве, заданном для конкретного датчика. При добавлении нового значения сверх лимита самое старое значение удаляется из истории

Файлы измерений содержат сырые показания датчиков. Каждое измерение записывается отдельным числом на новой строке:

```
21.4  
21.8  
22.1
```

Работу выполнить в два этапа внутри одной программы

Полноценный интерактивный CLI реализовывать не требуется. Достаточно подготовить в `main` понятный сценарий: создать профили калибровки, создать датчики, зарегистрировать их в контроллере, добавить серии измерений и вывести результат

На первом этапе реализовать класс истории измерений с собственным динамическим буфером. Буфер должен выделяться вручную при создании объекта и освобождаться при уничтожении объекта. Класс должен корректно вести себя при копировании, присваивании и перемещении

На втором этапе реализовать безопасную версию модели теплицы с RAII и стандартными средствами владения. Контроллер теплицы должен единолично владеть зарегистрированными датчиками. Общие профили калибровки должны разделяться между датчиками одной модели. Каталог профилей может хранить ссылки на существующие профили без продления их `lifetime`

В программе должны быть показаны ситуации:

- передача объекта в функцию по значению
- передача объекта в функцию по ссылке
- работа с указателем, который может быть пустым
- копирование истории измерений
- перемещение истории измерений
- уничтожение объектов при выходе из `scope`

Программа должна создать все датчики из таблицы, прочитать для них серии измерений и вывести:

- список датчиков
- последние измерения каждого датчика
- среднее значение по истории каждого датчика
- информацию о примененном профиле калибровки
- количество датчиков, которыми владеет контроллер

Требования

- Названия переменных, функций, типов и других сущностей программы должны быть осмысленными и записываться на английском языке без транслитерации
- Для истории измерений на первом этапе должен использоваться собственный динамический буфер через `new` и `delete`
- Класс с ручным динамическим буфером должен корректно реализовывать копирование, присваивание, перемещение и уничтожение объекта
- В программе должен быть явный пример перемещения объекта через `std::move`
- В основной модели теплицы должны использоваться `std::unique_ptr`, `std::shared_ptr` и `std::weak_ptr`

- Owning raw pointers во второй части работы использовать нельзя
- В программе должны быть примеры передачи объектов по значению и по ссылке
- В программе должна быть безопасная обработка пустого указателя
- Программа должна читать серии измерений из файлов, указанных в таблице датчиков
- Полноценный интерактивный CLI с циклом команд не требуется
- Программа должна быть грамотно разбита на модули и содержать как минимум один .hpp файл и два .cpp файла. Допускается использовать больше файлов
- Для заголовочных файлов использовать `#pragma once`
- Программа должна компилироваться и выполняться без ошибок
- Студент должен понимать написанный код, уметь обосновать свое решение и отвечать на вопросы во время защиты лабораторной работы

Минимальный набор операций:

- добавление измерения в историю с ограничением размера
- расчет среднего значения по истории измерений
- регистрация датчика в контроллере теплицы
- поиск профиля калибровки без продления его lifetime